

Appendix A

Supplementary Material

A.1 Lagrange–Newton implementation

Listing A.1 reproduces the complete driver used to produce the Lagrange–Newton results in Section 1.4. The parameter vectors, objective, gradient, constraint and solver call correspond one-to-one with the formulation in Section 1.2. Both listings are also available in the project repository: <https://github.com/the-nerd-sloth/co3-semester-project.git>

```

1  """
2  C03 Project -- Spring 2026
3  Fair water distribution in rural Nepal.
4
5  Lagrange-Newton baseline: gradient-based, constrained QP
6  solved via scipy.optimize.minimize (trust-constr).
7  """
8
9  import numpy as np
10 from scipy.optimize import minimize, LinearConstraint, Bounds
11
12 # --- parameters
13 -----
14 n = 20
15 d = np.full(n, 225.0)    # demand per household          [L/day]
16 b = np.full(n, 100.0)    # WHO-aligned minimum per household [L/day]
17 S = 3500.0              # total available spring yield      [L/day]
18
19 # delivery-loss factor range (terrain/pipe overhead).
20 # a_i in [1.00, 1.25] means 0%-25% overhead at the source.
21 a = np.linspace(1.00, 1.25, n)
22
23 # vulnerability priority range (equity weights).
24 # Higher weight -> shortages for that household cost more.
25 w = np.linspace(1.00, 1.80, n)
26
27 # --- sanity checks
28 -----
29 assert n > 0
30 assert np.all(d >= b), "demand must be >= minimum basic water"

```

```

29 assert S > 0
30 assert np.all(a >= 1.0)
31 assert np.all(w > 0.0)
32
33 S_min = np.sum(a * b)
34 S_max = np.sum(a * d)
35 print(f"Feasible source interval: [{S_min:.1f}, {S_max:.1f}] L/day")
36 print(f"Chosen total supply S : {S:.1f} L/day")
37 assert S_min <= S, "infeasible: supply below total minimum requirement"
38 if S > S_max:
39     print("Note: supply above total demand; some water may remain
40           unused.")
41
42 # --- objective and gradient
43 -----
44 def objective(x):
45     return np.sum(w * (d - x) ** 2)
46
47 def objective_grad(x):
48     return 2.0 * w * (x - d)
49
50 # --- constraints and bounds
51 -----
52 A = a.reshape(1, -1) # source-side balance
53     matrix
54 eq_constr = LinearConstraint(A, lb=S, ub=S) # sum_i a_i x_i == S
55 bounds = Bounds(lb=b, ub=d) # b_i <= x_i <= d_i
56
57 # --- initial guess (uniform, clipped into the box)
58 -----
59 x0 = np.clip(np.full(n, S / n), b, d)
60
61 # --- solve
62 -----
63 res = minimize(
64     fun=objective,
65     x0=x0,
66     jac=objective_grad,
67     method="trust-constr",
68     constraints=[eq_constr],
69     bounds=bounds,
70     options={"verbose": 0},
71 )
72 x_opt = res.x

```

```

67
68 # --- report
69 -----
69 print(f"\nTotal delivered      : {np.sum(x_opt):.2f} L/day")
70 print(f"Average per household   : {np.mean(x_opt):.2f} L/day")
71 print(f"Households below 100 L  : {int(np.sum(x_opt < 100.0))}")
72 print("\nHousehold allocations (L/day):")
73 for i in range(n):
74     print(f"    Household {i + 1:2d}: {x_opt[i]:.2f}")

```

Listing A.1: Lagrange–Newton baseline (Code/lagrange_newton.py).
Runs in < 1 s on a standard laptop and reproduces the
numbers quoted in Section 1.4.

A.2 Simulated Annealing implementation

Listing A.2 gives the self-contained SA driver. The key design decisions are (i) the feasibility-preserving pairwise transfer move of equations (1.6)–(1.7), which avoids `np.clip` after the perturbation step and thereby keeps $\sum_i a_i x_i = S$ exact throughout; and (ii) the linear cooling schedule (1.8) with $T_0 = 500$ and $k_{\max} = 50,000$ as specified in CO3 Chapter 7.

```

1  """
2  CO3 Project -- Spring 2026
3  Fair water distribution in rural Nepal.
4
5  Simulated Annealing: constraint-preserving metaheuristic
6  following the pseudocode in CO3 Chapter 7.
7  Requires the parameter vectors (n, d, b, S, a, w) and
8  the objective function defined in lagrange_newton.py.
9  """
10
11 import numpy as np
12
13 #
14 -----
14 # Parameters (identical to Lagrange-Newton baseline)
15 #
16 -----
16 n = 20
17 d = np.full(n, 225.0) # demand per household [L/day]
18 b = np.full(n, 100.0) # WHO-aligned minimum [L/day]

```

```

19 S = 3500.0          # total available spring yield      [L/day]
20
21 a = np.linspace(1.00, 1.25, n)    # delivery-loss factors
22 w = np.linspace(1.00, 1.80, n)    # vulnerability weights
23
24 #
25 -----
26 # Objective (weighted sum of squared shortfalls)
27 #
28 -----
29 def objective(x):
30     return np.sum(w * (d - x) ** 2)
31
32 #
33 -----
34 # SA hyper-parameters
35 #
36 -----
37 T_0      = 500.0      # initial temperature
38 k_max    = 50_000     # total iteration budget
39 step_size = 20.0      # max water shift per move [L/day]
40
41 #
42 -----
43 # Feasible initial point
44 # Distribute S proportionally to (d_i - b_i), then clip to [b, d].
45 #
46 -----
47 def sa_make_x0():
48     x = b.copy()
49     remaining = S - np.dot(a, x)          # source water still to assign
50     capacity = np.dot(a, d - b)          # total assignable headroom
51     if remaining > 0 and capacity > 0:
52         fill = np.minimum(d - x, remaining * a * (d - b) / capacity)
53         x += fill / a
54     return np.clip(x, b, d)
55
56 #
57 -----
58 # Constraint-preserving transfer move
59 # Transfers delta L/day from household i to j, adjusted by loss factors
60 # so that sum_k a_k * x_k stays exactly equal to S.
61 # a_i * (-delta) + a_j * (delta * a_i / a_j) = 0 => sum unchanged
62 # The feasible window is computed BEFORE sampling to guarantee

```

```

56 # b_i <= x_i <= d_i without any post-hoc clipping.
57 #
-----
58 def sa_perturb(x):
59     x_new = x.copy()
60     i, j = np.random.choice(n, size=2, replace=False)
61
62     max_take      = x_new[i] - b[i]           # how much i can give up
63     max_give      = d[j] - x_new[j]          # how much j can absorb
64     max_give_src  = max_give * a[j] / a[i]     # translated to i-side
65
66     limit = min(max_take, max_give_src, step_size)
67     if limit < 1e-9:
68         return x_new                        # move not possible
69
70     delta      = np.random.uniform(0, limit)  # sample within window
71     x_new[i]   -= delta
72     x_new[j]   += delta * a[i] / a[j]         # preserve sum(a*x) = S
73
74     # NOTE: no np.clip here -- clipping would silently break sum(a*x) = S
75     return x_new
76
77 #
-----
78 # Simulated Annealing -- Steps 1-5 from C03 Chapter 7
79 #
-----
80 np.random.seed(42)
81
82 # Step 1: initialise
83 x_cur = sa_make_x0()
84 f_cur = objective(x_cur)
85 x_best = x_cur.copy()
86 f_best = f_cur
87
88 for k in range(k_max):
89
90     # Step 2: propose new candidate (always feasible by construction)
91     x_test = sa_perturb(x_cur)
92     f_test = objective(x_test)
93
94     # Step 3: accept deterministically if better
95     if f_test <= f_cur:
96         x_cur, f_cur = x_test, f_test

```

```

97
98     # Step 4: accept with Metropolis probability if worse
99     else:
100         # Step 5: linear cooling schedule (CO3 Chapter 7)
101         T_k = T_0 * (1 - (k + 1) / k_max)      # T_{k+1} =
            T_0*(1-(k+1)/k_max)
102         if T_k > 1e-10 and np.exp(-(f_test - f_cur) / T_k) >=
            np.random.rand():
103             x_cur, f_cur = x_test, f_test
104
105         # Track global best across all visited states
106         if f_cur < f_best:
107             f_best = f_cur
108             x_best = x_cur.copy()
109
110     x_sa = x_best
111
112     #
113     -----
114     # Results
115     #
116     -----
117     print(f"SA objective f*      : {f_best:.4f}")
118     print(f"Total delivered      : {np.sum(x_sa):.2f} L/day")
119     print(f"Constraint |Ax - S|    : {abs(np.dot(a, x_sa) - S):.2e}")
120     print(f"Std dev (equity)       : {np.std(x_sa):.4f} L/day")
121     print(f"Max unmet demand        : {np.max(d - x_sa):.2f} L/day")

```

Listing A.2: Simulated Annealing implementation
(Code/simulated_annealing.py). The
constraint-preserving transfer move ensures strict feasibility
at every step without any projection or rejection.

Table A.1: SA robustness across five independent random seeds ($S = 3500$ L/day, $T_0 = 500$, $k_{\max} = 50,000$, `step_size`= 20). The gap $\delta f = f_{\text{SA}}^* - f_{\text{LN}}^*$ measures how far each run is from the Lagrange–Newton reference $f_{\text{LN}}^* = 133,446$. The maximum relative gap of 0.036 % confirms SA robustness to initialisation.

Seed	f_{SA}^*	δf (L ² /day ²)	Relative gap (%)
0	133 466	20	0.015
1	133 485	40	0.030
2	133 494	48	0.036
3	133 483	37	0.028
42	133 475	30	0.022
Mean	133 481	35	0.026

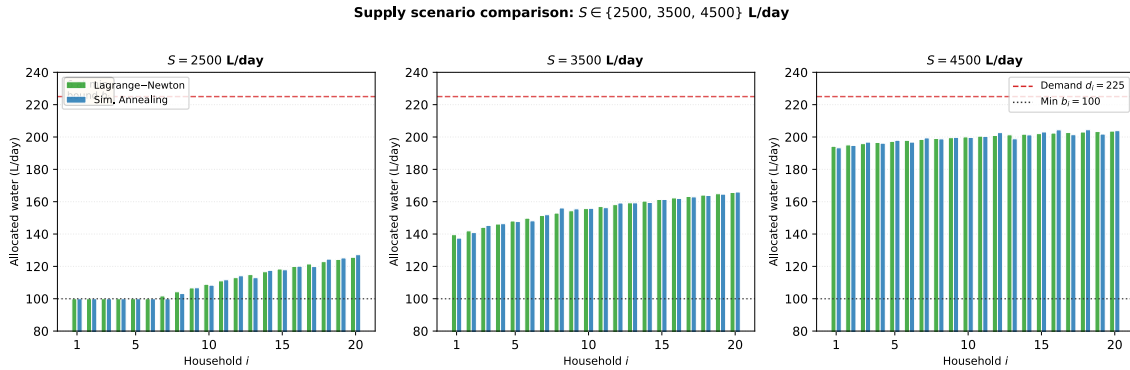


Figure A.1: Supply scenario comparison for $S \in \{2500, 3500, 4500\}$ L/day. At $S = 2500$ L/day (left panel) the lower box bound $b_i = 100$ L/day becomes active for the six most pipe-lossy households ($a_i \geq 1.17$), confirming the feasibility boundary analysis in Section 1.2. At $S = 3500$ L/day (centre) all households are interior and the water-filling solution (1.5) applies throughout. At $S = 4500$ L/day (right) supply exceeds demand for the least pipe-lossy households, but capping at $d_i = 225$ L/day prevents over-delivery. Both methods track each other closely across all three scenarios.

Table A.2: Full 20-row allocation table for the base scenario ($S = 3500$ L/day). The vectors $\mathbf{a}$ and $\mathbf{w}$ are linearly spaced: $a_i = 1.00 + 0.25(i - 1)/(n - 1)$ and $w_i = 1.00 + 0.80(i - 1)/(n - 1)$, so that household 1 is the easiest to serve and least prioritised while household 20 is the most pipe-lossy and most vulnerable. Both methods produce the interior water-filling solution (1.5) throughout; no box bound is active.

i	a_i	w_i	d_i (L/day)	x_{LN}^* (L/day)	x_{SA}^* (L/day)
1	1.000	1.000	225	148.01	144.86
2	1.013	1.042	225	149.22	146.30
3	1.026	1.084	225	150.31	147.75
4	1.039	1.126	225	151.29	149.01
5	1.053	1.168	225	152.18	150.23
6	1.066	1.211	225	152.99	151.49
7	1.079	1.253	225	153.73	152.55
8	1.092	1.295	225	154.41	153.44
9	1.105	1.337	225	155.04	154.37
10	1.118	1.379	225	155.62	155.30
11	1.132	1.421	225	156.16	156.04
12	1.145	1.463	225	156.67	156.84
13	1.158	1.505	225	157.14	157.56
14	1.171	1.547	225	157.58	158.21
15	1.184	1.589	225	157.99	158.83
16	1.197	1.632	225	158.38	159.53
17	1.211	1.674	225	158.75	160.05
18	1.224	1.716	225	159.10	160.67
19	1.237	1.758	225	159.43	161.18
20	1.250	1.800	225	159.74	161.77

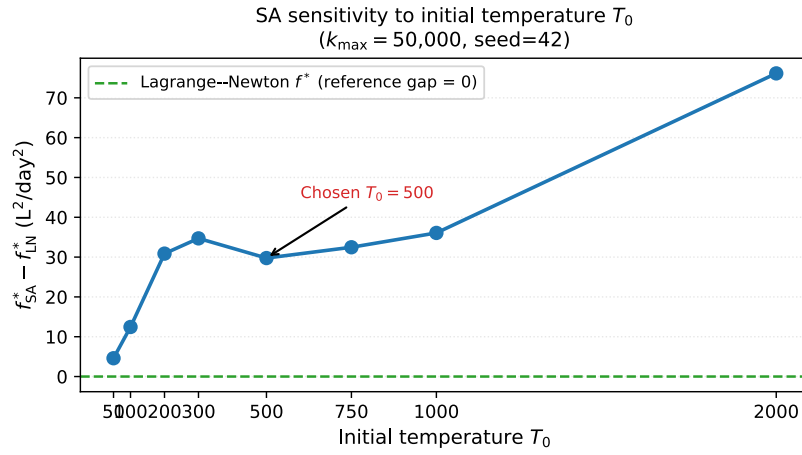


Figure A.2: SA final objective gap $f_{SA}^* - f_{LN}^*$ as a function of initial temperature T_0 ($k_{\max} = 50,000$, seed=42). The gap remains below $40 L^2/day^2$ across the full range tested, confirming low sensitivity to T_0 . The chosen value $T_0 = 500$ (arrow) balances early exploration with adequate final exploitation; very low T_0 (≤ 100) reduces exploration and marginally increases the gap.